

PROGRAMMATION FONCTIONNELLE 2 : TD2

janvier 2026 — Pierre Rousselin

On commence par travailler sur la conjonction logique : On rappelle sa définition dans le prélude de Rocq :

```
Inductive and (A B : Prop) : Prop :=
  conj (hA : A) (hB : B).
Notation "A /\ B" := (and A B) (at level 80, right associativity).
```

Exercice 1 : conjonction seule

1. En utilisant uniquement la tactique `exact` (et les constructions usuelles de Gallina, comme `match`, `fun`, `let`, etc) donner une preuve de chaque lemme suivant :
 - a) `Lemma foo1 (A B : Prop) (h : A /\ B) : B.`
 - b) `Lemma foo2 (A B : Prop) (hA : A) (hB : B) : A /\ (A /\ B).`
 - c) `Lemma and_comm (A B : Prop) (h : A /\ B) : B /\ A.`
 - d) `Lemma and_assoc' (A B C : Prop) (h : (A /\ B) /\ C) : A /\ (B /\ C).`
2. Faire de même en utilisant les tactiques `destruct` et/ou `split` et `exact`, mais seulement sous la forme `exact variable`.

--- * ---

Correction de l'exercice 1 :

```
1. Lemma foo1 (A B : Prop) (h : A /\ B) : B.
Proof.
  exact (match h with conj _ hb => hb end).
  (* exact (let 'conj _ hb := h in hb). *)
  (* exact (let (_, hb) := h in hb). *)
Qed.

Lemma foo2 (A B : Prop) (hA : A) (hB : B) : A /\ (A /\ B).
Proof.
  exact (conj hA (conj hA hB)).
Qed.

Lemma and_comm (A B : Prop) (h : A /\ B) : B /\ A.
Proof.
  exact (match h with conj ha hb => conj hb ha end).
  (* exact (let 'conj ha hb := h in conj hb ha). *)
  (* exact (let (ha, hb) := h in conj hb ha). *)
Qed.

Lemma and_assoc' (A B C : Prop) (h : (A /\ B) /\ C) : A /\ (B /\ C).
Proof.
  exact (
    match h with
    | conj hab hc => match hab with
      | conj ha hb => conj ha (conj hb hc)
    end
  end).
  (* exact (match h with conj (conj ha hb) hc => conj ha (conj hb hc) end). *)
  (* exact (let 'conj (conj ha hb) hc := h in conj ha (conj hb hc)). *)
  (* exact (let (hab, hc) := h in let (ha, hb) := hab in conj ha (conj hb hc)). *)
Qed.
```

2. **Lemma** foo1' (A B : Prop) (h : A /\ B) : B.

Proof.

```
destruct h as [_ hb]. exact hb.
```

Qed.

Lemma foo2' (A B : Prop) (hA : A) (hB : B) : A /\ (A /\ B).

Proof.

```
split.
```

```
- exact hA.
```

```
- split.
```

```
+ exact hA.
```

```
+ exact hB.
```

(* Avec assumption, on peut le faire en une ligne :

```
split; [| split]; assumption.
```

On devrait bientôt autoriser assumption.

*)

Qed.

Lemma and_comm' (A B : Prop) (h : A /\ B) : B /\ A.

Proof.

```
destruct h as [ha hb]. split.
```

```
- exact hb.
```

```
- exact ha.
```

(* Avec assumption :

```
destruct h; split; assumption.
```

(pas besoin de nommer si on n'utilise pas le nom) *)

Qed.

Lemma and_assoc'' (A B C : Prop) (h : (A /\ B) /\ C) : A /\ (B /\ C).

Proof.

```
destruct h as [[ha hb] hc].
```

```
split.
```

```
- exact ha.
```

```
- split.
```

```
+ exact hb.
```

```
+ exact hc.
```

(* Avec assumption :

```
destruct h as [[? ?] ?]; split; [| split]; assumption.
```

(pas besoin de nommer si on n'utilise pas le nom) *)

Qed.

--- * ---

Les règles de déduction naturelle qu'on considère sont les suivantes :

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge E-g)$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge I)$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (\wedge E-d)$$

$$\frac{}{\Gamma, A \vdash A} Ax$$

Ce sont celles vues en cours pour l'introduction et l'élimination de la conjonction, auxquelles on a ajouté l'axiome Ax qui dit qu'on peut prouver une proposition A qui apparaît déjà dans le contexte. D'ailleurs, ce contexte peut être considéré comme un ensemble, donc sans ordre particulier.

Exercice 2 : conjonction et déduction naturelle

1. Reprendre maintenant chacun des lemmes de l'exercice précédent et en donner une preuve sous la forme d'un arbre (dit *de dérivation*) utilisant uniquement les 4 règles précédentes. Par exemple, pour foo1, on voudrait un arbre dont la racine est

$$\overline{A \wedge B \vdash B}$$

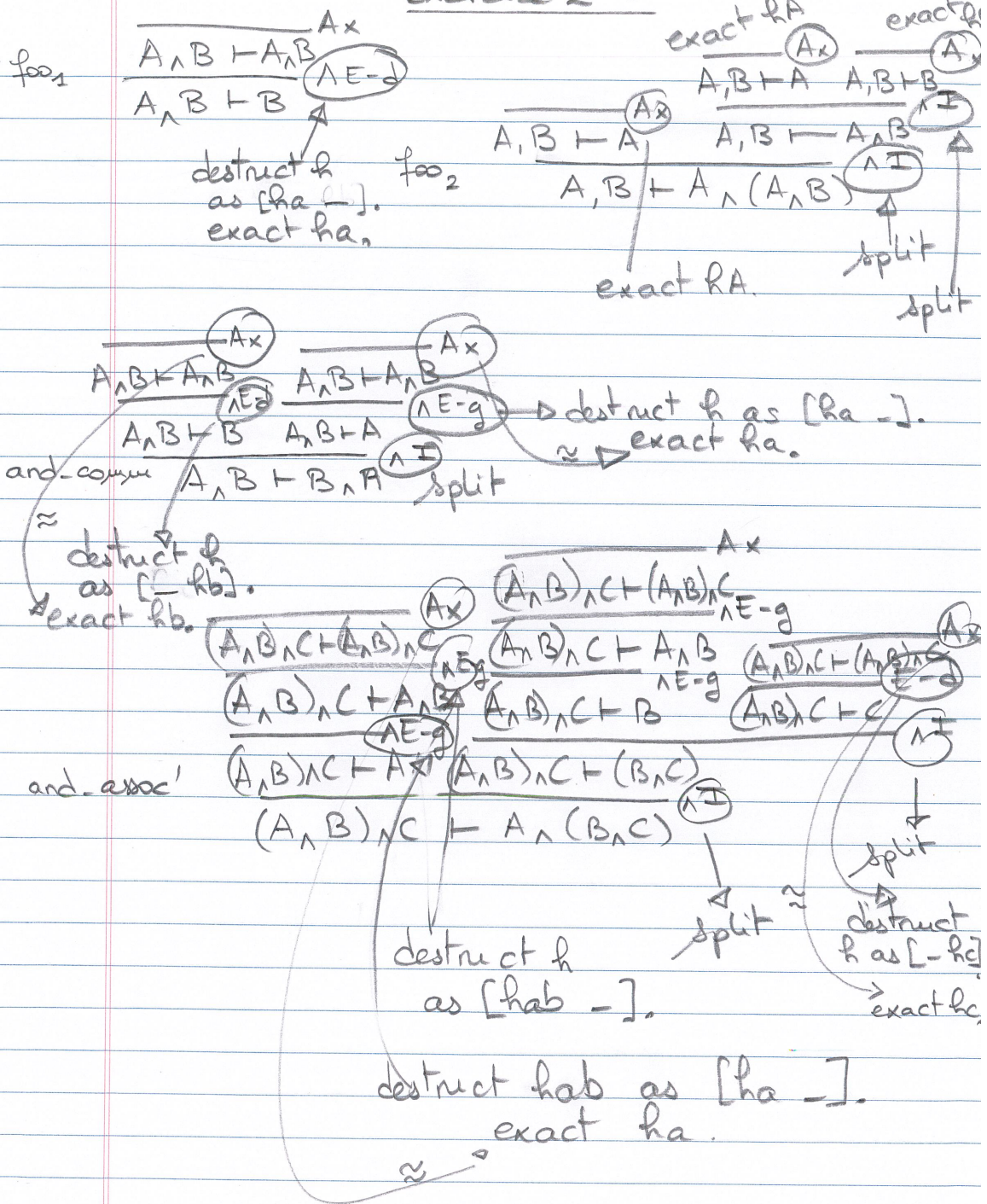
dont toutes les règles sont explicites et dont les feuilles sont des axiomes (donc ici, seulement Ax).

2. Comparer avec les preuves en Rocq avec **split** et **destruct**. Quelles sont les différences notables dans le raisonnement ? À quoi correspondent **split** et **destruct** ? À quoi correspond en Rocq la règle Ax ?

--- * ---

Correction de l'exercice 2 :

Exercice 2 TD2



--- * ---

On ajoute maintenant la disjonction :

```
Inductive or (A B : Prop) : Prop :=
  or_introl : (hA : A) | or_intror : (hB : B).
Notation "A \\/ B" := (or A B) (at level 85, right associativity).
```

Exercice 3 : disjonction (et éventuellement conjonction) en Rocq

- En utilisant uniquement la tactique `exact`, donner une preuve de chaque lemme suivant :
 - `Lemma bar (A B : Prop) (h : B) : A \\/ (A \\/ B).`
 - `Lemma or_comm (A B : Prop) (h : A \\/ B) : B \\/ A.`
 - `Lemma and_or_distr_l (A B C : Prop) (h : A /\ (B \\/ C)) : (A /\ B) \\/ (A /\ C).`
- Faire de même en utilisant les tactiques `destruct`, `left`, `split`, `right` et `exact`, mais seulement sous la forme `exact variable`.

--- * ---

Correction de l'exercice 3 :

- `Lemma bar (A B : Prop) (h : B) : A \\/ (A \\/ B).`
`Proof.`
`exact (or_intror (or_intror h)).`
`Qed.`

`Lemma or_comm (A B : Prop) (h : A \\/ B) : B \\/ A.`
`Proof.`
`exact (`
`match h with`
`| or_introl ha => or_intror ha`
`| or_intror ha => or_introl ha`
`end).`
`Qed.`

`Lemma and_or_distr_l (A B C : Prop) (h : A /\ (B \\/ C)) : (A /\ B) \\/ (A /\ C).`
`Proof.`
`exact (`
`let 'conj ha hborc := h in`
`match hborc with`
`| or_introl hb => or_introl (conj ha hb)`
`| or_intror hc => or_intror (conj ha hc)`
`end).`
`Qed.`

`2. Lemma bar' (A B : Prop) (h : B) : A \\/ (A \\/ B).`
`Proof.`
`right. right. exact h.`
`Qed.`

`Lemma or_comm' (A B : Prop) (h : A \\/ B) : B \\/ A.`
`Proof.`
`(* destruct h as [h | h]; [right | left]; exact h. *)`
`destruct h as [ha | hb].`
`- right. exact ha.`
`- left. exact hb.`
`Qed.`

`Lemma and_or_distr_l' (A B C : Prop) (h : A /\ (B \\/ C)) : (A /\ B) \\/ (A /\ C).`

Proof.

```
(* destruct h as [h [h1 | h2]]; [left | right]; split; assumption. *)
destruct h as [ha [hb | hc]].
- left. split.
  + exact ha.
  + exact hb.
- right. split.
  + exact ha.
  + exact hc.
Qed.
```

--- * ---

On ajoute les règles de déduction naturelles suivantes.

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} (\vee\text{-g}) \qquad \frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash P \quad \Gamma, B \vdash P}{\Gamma \vdash P} (\vee\text{-E})$$

$$\frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} (\vee\text{-d})$$

Exercice 4 : déduction naturelle et disjonction

En utilisant ces nouvelles règles, donner des arbres de dérivation pour `bar` et `or_comm`.

--- * ---

Correction de l'exercice 4 :

Exercice 4 TD2

$$\text{bar} \quad \frac{\frac{\frac{}{Ax} \quad B \vdash B}{B \vdash A \vee B} (VI-d)}{B \vdash A \vee (A \vee B)} (VI-d)$$

$$\text{or-comm} \quad \frac{\frac{Ax \quad \frac{}{Ax} \quad A \vdash A}{A \vee B \vdash A \vee B} \quad \frac{\frac{}{Ax} \quad A \vdash A}{A \vdash B \vee A} (VI-d) \quad \frac{\frac{}{Ax} \quad B \vdash B}{B \vdash B \vee A} (VI-g)}{A \vee B \vdash B \vee A}$$

$$\begin{array}{c} \frac{\frac{}{Ax} \quad A \wedge (B \vee C), B \vdash A \wedge (B \vee C)}{\Gamma, B \vdash A} \wedge\text{-Eg} \quad \frac{\frac{}{Ax} \quad A \wedge (B \vee C), C \vdash A \wedge (B \vee C)}{\Gamma, C \vdash A} \wedge\text{-Eg} \\ \frac{\Gamma, B \vdash A \quad \Gamma, B \vdash B}{\Gamma, B \vdash A \wedge B} \wedge\text{-I} \quad \frac{\Gamma, C \vdash A \quad \Gamma, C \vdash C}{\Gamma, C \vdash A \wedge C} \wedge\text{-I} \\ \frac{A \wedge (B \vee C) \vdash B \vee C \quad \Gamma, B \vdash F}{\Gamma, B \vdash F} \vee\text{-g} \quad \frac{\Gamma, C \vdash F}{\Gamma, C \vdash F} \vee\text{-g} \\ \text{and-or-dist-e} \quad \frac{\frac{\Gamma \quad A \wedge (B \vee C) \vdash B \vee C}{\Gamma} \quad \frac{\frac{\Gamma \quad A \wedge B}{F} \vee \frac{\frac{\Gamma \quad A \wedge C}{F}}{F}}{\Gamma} \end{array}$$

--- * ---

Exercice 5 : Implication et termes de preuve

On rappelle que, si A et B sont deux propositions, on peut voir $A \rightarrow B$ à la fois comme le type des fonctions de A vers B et comme « A implique B ». Prouver $A \rightarrow B$ peut alors se faire en construisant une fonction et utiliser une preuve h de $A \rightarrow B$ revient à appliquer cette fonction h à un élément de type A .

1. En utilisant uniquement `exact` et des définitions/applications de fonctions, prouver les lemmes suivants :
 - a) `Lemma baz1 (A : Prop) : A -> A.`
 - b) `Lemma baz2 (A B C : Prop) (hA : A) (h : A -> B) (h' : B -> C) : C.`
 - c) `Lemma baz3 (A B : Prop) : A -> B -> A.`
 - d) `Lemma baz4 (A B C : Prop) (hab : A -> B) (hbc : B -> C) : A -> C.`
2. Reprendre les preuves des lemmes précédents, mais en utilisant les tactiques `intros` et/ou `apply`.

--- * ---

Correction de l'exercice 5 :

```

— Lemma baz1 (A : Prop) : A -> A.
Proof.
  exact (fun h => h).
Qed.

Lemma baz2 (A B C : Prop) (hA : A) (h : A -> B) (h' : B -> C) : C.
Proof.
  exact (h' (h hA)).
Qed.

Lemma baz3 (A B : Prop) : A -> B -> A.
Proof.
  exact (fun hA => (fun _ => hA)).
  (* exact (fun hA _ => hA). *)
Qed.

Lemma baz4 (A B C : Prop) (hab : A -> B) (hbc : B -> C) : A -> C.
Proof.
  exact (fun ha => hbc (hab ha)).
Qed.

— Lemma baz1' (A : Prop) : A -> A.
Proof.
  intros hA. exact hA.
Qed.

Lemma baz2' (A B C : Prop) (hA : A) (h : A -> B) (h' : B -> C) : C.
Proof.
  apply h'. apply h. exact hA.
  (* apply h', h; exact hA. *)
Qed.

Lemma baz3' (A B : Prop) : A -> B -> A.
Proof.
  intros hA _. exact hA.
  (* exact (fun hA _ => hA). *)
Qed.

Lemma baz4' (A B C : Prop) (hab : A -> B) (hbc : B -> C) : A -> C.

```



```
Proof.
  intros hA. apply hbc. apply hab. exact hA.
  (* intros hA. apply hbc, hab. exact hA. *)
Qed.
```

--- * ---

On ajoute finalement ces deux règles pour l'implication :

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} (\rightarrow\text{-I}) \qquad \frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} (\rightarrow\text{-E})$$

Exercice 6 : Dédution naturelle et implication

Donner un arbre de dérivation pour chaque lemme de l'exercice précédent.

--- * ---

Correction de l'exercice 6 :

TD2 exercice 6

$$\text{baz}_1 \quad \frac{\frac{}{A \vdash A} A_x}{\vdash A \rightarrow A} \rightarrow\text{-I}$$

$$\text{baz}_2 \quad \frac{\frac{}{A \vdash B \rightarrow C} A_x \quad \frac{\frac{}{\Gamma \vdash A \rightarrow B} A_x \quad \frac{}{\Gamma \vdash A} A_x}{\Gamma \vdash B} \rightarrow\text{-E}}{\underbrace{A, A \rightarrow B, B \rightarrow C}_{\vdash} \vdash C} \rightarrow\text{-E}$$

$$\text{baz}_3 \quad \frac{\frac{\frac{}{A, B \vdash A} A_x}{A \vdash B \rightarrow A} \rightarrow\text{-I}}{\vdash A \rightarrow (B \rightarrow B)} \rightarrow\text{-I}$$

$$\text{baz}_4 \quad \frac{\frac{}{A, A \rightarrow B, B \rightarrow C \vdash C} \text{voir baz}_2}{A \rightarrow B, B \rightarrow C \vdash A \rightarrow C} \rightarrow\text{-I}$$

--- * ---